



Générateur de nombres pseudo-aléatoires cryptographiquement sûr (CSPRNG) basé sur AES-128 en mode CTR



Réalisé par :

Bouhadfane Elbatoul

Encadré par:

Dr Bouchoucha Lydia

	1
1. Introduction	5
2. Objectifs du rapport	5
3. Rappels sur les nombres pseudo-aléatoires	5
3.1 Définition	5
3.2 Caractéristiques d'un bon PRNG	6
3.3 Limites des PRNG classiques	6
4. Présentation de l'algorithme AES	7
4.1 Définition	7
4.2 Caractéristiques	7
4.3 Étapes principales d'AES	7
5. Principe du PRNG basé sur AES	8
5.1 Idée principale	8
5.2 Schéma général	8
5.3 Fonctionnement	9
6. Architecture du générateur AES-PRNG	9
6.1 Composants	9
7. Implémentation du générateur AES-128 CSPRNG	10
7.1 Implémentation de l'algorithme AES	10
7.2 Utilisation du mode CTR	11
7.3 Génération sécurisée du seed	12
7.4 Structure interne du générateur	12
7.5 Mécanisme de reseeding	13
7.6 Validation du générateur	13
8. Description fonctionnelle	14
9. Analyse de sécurité	14
9.1 Avantages	14
9.2 Propriétés de sécurité	14
9.3 Attaques possibles	15
10. Performances	15
10.1 Avantages	15
10.2 Inconvénients	16
11. Applications	16
12. Validation du générateur (Tests statistiques)	17
12.1 Test de fréquence (Monobit Test)	17
12.2 Test des runs	17
12.3 Résultats	17
Conclusion	18
Références	19

1. Introduction

La génération de nombres pseudo-aléatoires (PRNG – *Pseudo Random Number Generator*) constitue un élément essentiel dans de nombreux domaines de l'informatique moderne, notamment la cryptographie, la sécurité des systèmes, les simulations numériques et les systèmes distribués. Ces générateurs produisent des suites de nombres qui paraissent aléatoires, mais qui sont en réalité déterministes, car elles sont entièrement définies à partir d'une valeur initiale appelée **graine (seed)**.

Dans la pratique, la qualité d'un générateur pseudo-aléatoire est cruciale, en particulier dans les applications de sécurité. En effet, les PRNG classiques présentent souvent des faiblesses en termes de prévisibilité, ce qui les rend vulnérables aux attaques cryptographiques. Pour répondre à ces exigences de sécurité, on utilise des générateurs dits **cryptographiquement sûrs (CSPRNG – Cryptographically Secure Pseudo Random Number Generator)**.

Dans ce contexte, des algorithmes de chiffrement symétrique robustes tels que **AES (Advanced Encryption Standard)** sont fréquemment exploités afin de concevoir des générateurs de nombres pseudo-aléatoires sécurisés. Grâce à ses propriétés de diffusion, de confusion et de résistance aux attaques connues, AES constitue une base solide pour la construction de PRNG cryptographiquement sûrs.

2. Objectifs du rapport

Ce rapport a pour principaux objectifs de :

- Comprendre le principe général des générateurs de nombres pseudo-aléatoires (PRNG) et leur importance en informatique.
- Étudier en détail l'algorithme de chiffrement **AES (Advanced Encryption Standard)**.
- Expliquer comment l'algorithme AES peut être détourné et utilisé comme base pour un générateur de nombres pseudo-aléatoires.
- Analyser les aspects de sécurité ainsi que les performances d'un PRNG basé sur AES.
- Présenter une architecture claire et structurée d'un générateur pseudo-aléatoire fondé sur AES.

3. Rappels sur les nombres pseudo-aléatoires

3.1 Définition

Un nombre pseudo-aléatoire est un nombre généré par un algorithme **déterministe** capable de produire une suite de valeurs qui imitent un comportement aléatoire. Contrairement à

l'aléatoire véritable, ces nombres ne sont pas réellement imprévisibles, car ils dépendent entièrement d'une valeur initiale appelée **graine (seed)**.

Ainsi, pour une même graine de départ, le générateur produira toujours la même séquence de nombres, ce qui rend le processus reproductible mais pas véritablement aléatoire.

3.2 Caractéristiques d'un bon PRNG

Un générateur de nombres pseudo-aléatoires efficace doit posséder plusieurs propriétés essentielles :

- **Rapidité d'exécution** : il doit être capable de générer des nombres rapidement pour répondre aux besoins des applications en temps réel.
- **Longue période** : la séquence générée doit être très longue avant de commencer à se répéter.
- **Imprévisibilité** : il doit être difficile de prédire les prochains nombres à partir des valeurs précédentes.
- **Reproductibilité** : avec la même graine (seed), la même suite de nombres doit être obtenue, ce qui est utile pour le débogage et les simulations.
- **Résistance cryptographique (CSPRNG)** : dans le cas d'applications de sécurité, le générateur doit résister aux attaques visant à reconstruire la séquence ou l'état interne.

3.3 Limites des PRNG classiques

Les générateurs pseudo-aléatoires classiques présentent plusieurs faiblesses importantes qui limitent leur utilisation en cryptographie :

- **Linear Congruential Generator (LCG)** : bien qu'il soit simple et rapide, ce générateur est facilement prévisible si certains paramètres ou sorties sont connus.
- **Mersenne Twister** : très performant pour les simulations, il n'est cependant pas adapté à la cryptographie, car son état interne peut être reconstruit à partir d'un nombre suffisant de sorties.
- **Sensibilité à l'état interne** : dans la majorité des PRNG classiques, la connaissance partielle de l'état interne peut permettre de prédire les valeurs futures, ce qui constitue une faille majeure en sécurité.

4.Présentation de l'algorithme AES

4.1 Définition

AES (Advanced Encryption Standard) est un algorithme de chiffrement symétrique adopté comme standard par le NIST (*National Institute of Standards and Technology*) en 2001. Il est aujourd'hui l'un des algorithmes de chiffrement les plus utilisés dans le monde pour assurer la confidentialité des données.

AES repose sur une clé secrète unique utilisée à la fois pour le chiffrement et le déchiffrement, ce qui en fait un algorithme symétrique.

4.2 Caractéristiques

L'algorithme AES possède plusieurs caractéristiques essentielles qui expliquent sa robustesse et sa popularité :

- **Taille de bloc** : 128 bits (les données sont traitées par blocs de 128 bits)
- **Tailles de clé** : 128 bits, 192 bits ou 256 bits selon le niveau de sécurité souhaité
- **Structure interne** : AES est basé sur un réseau de substitution-permutation (SPN – *Substitution Permutation Network*)
- **Nombre de tours de chiffrement** :
 - 10 tours pour AES-128
 - 12 tours pour AES-192
 - 14 tours pour AES-256

Chaque augmentation de la taille de clé améliore le niveau de sécurité, mais augmente également le coût computationnel.

4.3 Étapes principales d'AES

Chaque cycle de chiffrement (appelé tour) dans AES est constitué de plusieurs transformations appliquées sur l'état des données :

- **SubBytes** : opération de substitution non linéaire où chaque octet est remplacé par un autre via une S-Box, ce qui apporte la confusion.
- **ShiftRows** : permutation des lignes de la matrice d'état afin de disperser les données.
- **MixColumns** : transformation mathématique appliquée aux colonnes pour mélanger les données et assurer une bonne diffusion.
- **AddRoundKey** : opération XOR entre l'état courant et une sous-clé dérivée de la clé principale.

Ces étapes sont répétées plusieurs fois selon le nombre de tours défini par la taille de la clé, garantissant ainsi un niveau élevé de sécurité cryptographique.

5. Principe du PRNG basé sur AES

5.1 Idée principale

L'idée fondamentale d'un générateur de nombres pseudo-aléatoires basé sur AES repose sur l'utilisation de l'algorithme de chiffrement AES comme une fonction de génération de sortie pseudo-aléatoire.

Dans ce contexte, AES ne sert plus uniquement à chiffrer des données, mais à produire des blocs de données apparemment aléatoires en chiffrant une entrée variable (appelée compteur ou état interne) avec une clé secrète.

Ainsi, la sécurité du générateur dépend directement de la robustesse d'AES et du secret de la clé utilisée.

```
block = aes_encrypt_block(self._counter_bytes(), self._round_keys)

self._increment_counter()
```

En pratique, cette opération est implémentée comme une fonction de chiffrement AES appliquée sur un compteur incrémental.

5.2 Schéma général

Le fonctionnement d'un PRNG basé sur AES repose sur trois éléments principaux :

- Une **clé secrète K**, connue uniquement du système.
- Un **compteur initial C₀**, souvent dérivé d'un seed.
- Une fonction de chiffrement AES.

Le principe est le suivant :

- Chaque valeur du compteur est chiffrée :

AES(K, C_i) → sortie pseudo-aléatoire

- Après chaque génération, le compteur est incrémenté :

C_{i+1} = C_i + 1

Ce processus permet de produire une suite de valeurs pseudo-aléatoires uniques.

5.3 Fonctionnement

Le fonctionnement du générateur AES-PRNG se déroule en deux étapes principales :

Initialisation

- Choisir une clé AES secrète **K**
- Initialiser le compteur **C** avec une valeur de départ (seed)

Génération des nombres pseudo-aléatoires

Ensuite, les étapes suivantes sont répétées pour générer la suite :

- $X_1 = \text{AES}(K, C)$
- Incrémentation du compteur : $C = C + 1$
- $X_2 = \text{AES}(K, C)$
- Et ainsi de suite...

Chaque sortie X_i correspond à un bloc de **128 bits**, qui est utilisé comme un nombre pseudo-aléatoire.

6. Architecture du générateur AES-PRNG

6.1 Composants

L'architecture d'un générateur de nombres pseudo-aléatoires basé sur AES repose sur plusieurs éléments essentiels qui interagissent pour produire une suite de valeurs sécurisées :

- **Clé secrète AES (K)** : elle constitue l'élément principal de sécurité du système. Sans cette clé, il est impossible de reproduire ou prédire la suite générée.
- **Compteur ou vecteur d'état (C)** : il représente l'entrée variable de l'algorithme. Il est initialisé à partir d'un seed et évolue à chaque génération.
- **Module AES (chiffrement)** : c'est le cœur du système, chargé de transformer le compteur en sortie pseudo-aléatoire à l'aide de la clé secrète.
- **Buffer de sortie** : il stocke les valeurs générées afin de les utiliser dans les applications (cryptographie, simulation, etc.).

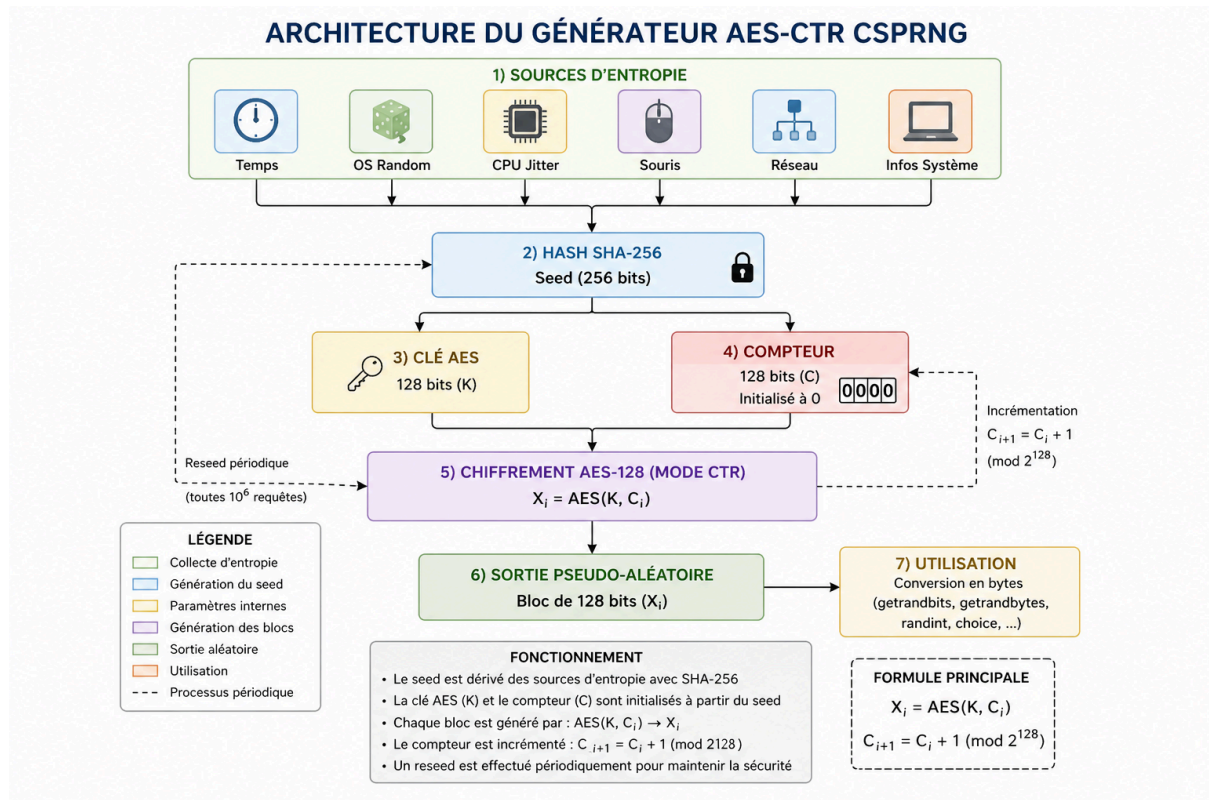


Figure 1 : Architecture du générateur AES-CTR CSPRNG

7. Implémentation du générateur AES-128 CSPRNG

7.1 Implémentation de l'algorithme AES

Dans ce projet, l'algorithme AES-128 a été entièrement implémenté manuellement sans utiliser de bibliothèques cryptographiques externes. Cette approche permet de mieux comprendre le fonctionnement interne de l'algorithme.

Les différentes transformations d'AES ont été programmées, notamment :

- SubBytes (substitution via S-Box)
- ShiftRows (décalage des lignes)
- MixColumns (opérations dans le corps $\text{GF}(2^8)$)
- AddRoundKey (XOR avec la clé)
- Key Expansion (génération des sous-clés avec Rcon)

```
def _sub_bytes(state: bytearray) -> bytearray:
    """
    SubBytes  apply the AES S-Box to every byte of the 4x4 state matrix.
    The state is stored as a flat 16-byte array in column-major order.
    """
    return bytearray(AES_SBOX[b] for b in state)
```

Cette fonction applique la substitution non linéaire S-Box de l'algorithme AES.

7.2 Utilisation du mode CTR

Le générateur repose sur le mode compteur (CTR – Counter Mode). Dans ce mode, chaque bloc de sortie est généré en chiffrant une valeur de compteur avec une clé secrète.

Le principe est le suivant :

- Une clé secrète K est utilisée
- Un compteur C est initialisé
- À chaque étape : une sortie est produite en appliquant AES sur le compteur
- Le compteur est ensuite incrémenté

Ce mécanisme garantit l'unicité des entrées et donc des sorties.

```
def generate_block(self) -> bytes:
    """
    Generate one 128-bit (16-byte) random output block.

    Formula: block = AES_encrypt(key, counter); counter++
    """
    if self._block_count >= self.RESEED_INTERVAL:
        self.reseed() # mandatory periodic reseed

    block = aes_encrypt_block(self._counter_bytes(), self._round_keys)
    self._increment_counter()
    self._block_count += 1
    return block
```

Chaque bloc généré correspond au chiffrement du compteur avec AES, puis incrémentation du compteur.

Chaque appel au générateur produit un bloc de 128 bits.

Seed (256 bits)

└─ Key (128 bits)

└─ Counter (128 bits)

Cette séparation permet de dériver indépendamment la clé cryptographique et l'état interne du générateur.

7.5 Mécanisme de reseeding

Afin de garantir la sécurité sur le long terme, le générateur intègre un mécanisme de reseeding. Après un certain nombre de blocs générés, une nouvelle entropie est collectée et utilisée pour réinitialiser la clé et le compteur.

Ce mécanisme permet de limiter les risques liés à une utilisation prolongée du générateur

7.6 Validation du générateur

Afin d'évaluer la qualité du générateur, des tests statistiques basés sur la norme NIST SP 800-22 ont été utilisés.

Deux tests principaux ont été appliqués :

- Test de fréquence (Monobit Test)
- Test des runs

Ces tests permettent de vérifier que la distribution des bits générés est proche d'un comportement aléatoire. Les résultats obtenus montrent que le générateur passe les tests de base, ce qui confirme la qualité de l'approche utilisée.

8. Description fonctionnelle

Le fonctionnement global du générateur AES-PRNG peut être décrit de manière simple comme suit. Le système démarre à partir d'une valeur initiale appelée seed, à partir de laquelle sont dérivées une clé secrète K ainsi qu'une valeur initiale du compteur C.

Ensuite, le compteur est utilisé comme entrée de l'algorithme AES avec la clé K. À chaque étape, l'opération de chiffrement AES(K, C) produit une sortie notée X_i , qui correspond à un nombre pseudo-aléatoire.

Après chaque génération, le compteur est automatiquement incrémenté ($C = C + 1$) afin de garantir que chaque entrée fournie à AES est différente.

Ce mécanisme est répété de manière itérative, ce qui permet de produire une suite de valeurs pseudo-aléatoires déterministe mais présentant un comportement statistiquement aléatoire.

$$X_1 = \text{AES}(K, C)$$

$$C = C + 1$$

$$X_2 = \text{AES}(K, C)$$

Ce processus est répété de manière itérative pour générer une suite déterministe mais imprévisible.

9. Analyse de sécurité

9.1 Avantages

L'utilisation de l'algorithme AES dans la conception d'un générateur de nombres pseudo-aléatoires présente plusieurs avantages majeurs en matière de sécurité :

- **Résistance cryptographique élevée** : AES est un algorithme largement étudié et considéré comme sécurisé contre les attaques connues lorsqu'il est correctement implémenté.
- **Non prédictibilité sans la clé** : sans connaissance de la clé secrète, il est pratiquement impossible de prédire les sorties générées.
- **Basé sur un standard mondial sécurisé** : AES est un standard adopté par le NIST et utilisé dans de nombreux protocoles de sécurité (TLS, VPN, etc.).
- **Grande période** : la taille du compteur et du bloc AES (128 bits) permet une très grande période de génération (jusqu'à 2^{128} valeurs possibles), ce qui empêche les répétitions à court terme.

9.2 Propriétés de sécurité

Un générateur de nombres pseudo-aléatoires basé sur AES (AES-PRNG) est considéré comme sûr uniquement si certaines conditions sont respectées :

- **La clé secrète doit rester confidentielle** : si la clé est compromise, toute la suite générée peut être reconstruite.
- **Le compteur ne doit jamais se répéter** : chaque valeur d'entrée doit être unique pour éviter toute répétition de sortie.

- **AES doit rester résistant aux attaques cryptographiques connues** : la sécurité du système repose entièrement sur la robustesse de l'algorithme AES.

Lorsque ces conditions sont respectées, le générateur peut être utilisé dans des applications cryptographiques sensibles.

9.3 Attaques possibles

Malgré sa robustesse, un AES-PRNG peut devenir vulnérable si son implémentation est incorrecte ou mal sécurisée :

- **Attaque par force brute sur la clé** : un attaquant peut tenter toutes les clés possibles, mais cela reste extrêmement difficile avec des clés de 128 bits ou plus.
- **Mauvaise gestion du seed** : un seed faible ou prévisible peut réduire fortement la sécurité du générateur.
- **Réutilisation du compteur (grave erreur)** : si une même valeur de compteur est utilisée plusieurs fois avec la même clé, cela peut conduire à la répétition des sorties, compromettant totalement la sécurité du système.

10. Performances

10.1 Avantages

L'utilisation de l'algorithme AES dans un générateur de nombres pseudo-aléatoires présente des performances intéressantes, notamment dans les environnements modernes :

- **Très rapide sur les processeurs modernes** : AES est optimisé pour fonctionner efficacement sur les architectures actuelles, ce qui permet une génération rapide des valeurs pseudo-aléatoires.
- **Optimisation matérielle (AES-NI)** : de nombreux processeurs intègrent des instructions matérielles dédiées (AES-NI) qui accélèrent considérablement les opérations de chiffrement AES.
- **Génération efficace de blocs de 128 bits** : chaque appel à AES produit directement un bloc de 128 bits, ce qui rend la génération de données efficace et structurée.

L'utilisation d'un pool d'entropie combiné avec une fonction de hachage SHA-256 permet de renforcer considérablement la sécurité du générateur en rendant le seed difficilement prévisible.

10.2 Inconvénients

Malgré ses avantages, un PRNG basé sur AES présente également certaines limitations en termes de performance :

- **Plus lent que les PRNG classiques** : comparé à des générateurs simples comme LCG ou Mersenne Twister, AES nécessite davantage de calculs, ce qui peut réduire la vitesse de génération dans certains contextes non sécurisés.
- **Dépendance au matériel pour performance optimale** : les meilleures performances sont obtenues uniquement sur des processeurs supportant les instructions AES-NI. Sans cette optimisation matérielle, les performances peuvent être plus faibles.

11. Applications

Les générateurs de nombres pseudo-aléatoires basés sur AES sont largement utilisés dans plusieurs domaines critiques où la sécurité et la qualité de l'aléatoire sont essentielles :

- **Cryptographie (TLS, VPN)** : ils sont utilisés pour générer des clés de session, des nonces et des vecteurs d'initialisation dans les protocoles de sécurité comme TLS et les réseaux privés virtuels (VPN), garantissant la confidentialité et l'intégrité des communications.
- **Jeux vidéo (aléatoire sécurisé)** : dans certains jeux en ligne, un PRNG sécurisé est utilisé pour éviter les manipulations et garantir l'équité (loot, tirages, événements aléatoires).
- **Simulations scientifiques** : les générateurs pseudo-aléatoires servent à modéliser des phénomènes complexes (physique, statistiques, finance) où une bonne distribution des valeurs est nécessaire.
- **Génération de clés cryptographiques** : AES-PRNG est utilisé pour produire des clés cryptographiques robustes, difficiles à prédire ou à reproduire.
- **Protocoles de sécurité** : ils interviennent dans divers mécanismes de sécurité comme l'authentification, les signatures numériques et les systèmes de protection des données.

Comparaison avec d'autres PRNG

Générateur	Sécurité	Vitesse	Usage cryptographique
LCG (Linear Congruential Generator)	Faible	Très rapide	Non

Mersenne Twister	Moyenne	Rapide	Non
AES-PRNG	Très élevée	Moyenne	Oui

Interprétation

Les générateurs classiques comme LCG et Mersenne Twister sont efficaces pour les simulations, mais ils ne sont pas adaptés à la cryptographie. En revanche, AES-PRNG offre un niveau de sécurité très élevé, ce qui le rend adapté aux applications sensibles.

12. Validation du générateur (Tests statistiques)

Afin d'évaluer la qualité du générateur pseudo-aléatoire, des tests statistiques inspirés de la norme NIST SP 800-22 ont été réalisés.

```
p_value = math.erfc(S_obs / math.sqrt(2))
```

Un p-value supérieur à 0.01 indique que la séquence est statistiquement acceptable selon le test.

12.1 Test de fréquence (Monobit Test)

Ce test vérifie si le nombre de bits à 1 et à 0 est approximativement équilibré dans la séquence générée.

Un générateur aléatoire doit produire environ 50 % de bits à 1 et 50 % de bits à 0. Le test calcule une p-value permettant de déterminer si la séquence est acceptable.

12.2 Test des runs

Ce test analyse le nombre de séquences consécutives de bits identiques (runs). Il permet de détecter si les bits alternent trop rapidement ou pas suffisamment.

12.3 Résultats

Les résultats obtenus montrent que le générateur passe les tests statistiques de base, ce qui indique un comportement conforme à celui d'un générateur aléatoire fiable.

Conclusion

La génération de nombres pseudo-aléatoires joue un rôle fondamental dans de nombreux domaines de l'informatique moderne, en particulier en cryptographie et en sécurité des systèmes. Cependant, les générateurs classiques présentent souvent des limites importantes en termes de prévisibilité et de résistance aux attaques.

Dans ce contexte, l'utilisation de l'algorithme AES (Advanced Encryption Standard) comme base d'un générateur de nombres pseudo-aléatoires constitue une solution robuste et fiable. Grâce à ses propriétés cryptographiques éprouvées, AES permet de construire un PRNG sécurisé capable de produire des séquences de valeurs imprévisibles, à condition que la clé secrète soit correctement protégée et que le compteur ne soit jamais réutilisé.

L'étude réalisée dans ce rapport montre que le PRNG basé sur AES offre un excellent niveau de sécurité, une grande période de génération et une bonne adaptabilité aux applications critiques telles que TLS, VPN, la génération de clés cryptographiques et les systèmes de sécurité avancés. Toutefois, cette approche présente un coût en performance légèrement supérieur aux générateurs classiques, bien que celui-ci soit largement compensé par le gain en sécurité, notamment grâce aux optimisations matérielles comme AES-NI.

En conclusion, le PRNG basé sur AES représente une solution moderne, efficace et hautement sécurisée, adaptée aux environnements où la confidentialité et l'intégrité des données sont des exigences essentielles.

Références

Ouvrages et standards fondamentaux

- NIST. *FIPS PUB 197: Advanced Encryption Standard (AES)*, National Institute of Standards and Technology, 2001.
→ Document officiel définissant l'algorithme AES comme standard de chiffrement symétrique.
- Menezes, A. J., van Oorschot, P. C., Vanstone, S. A.
Handbook of Applied Cryptography, CRC Press, 1996.
→ Référence majeure en cryptographie appliquée couvrant les PRNG, les fonctions de hachage et les systèmes de chiffrement.
- Kelsey, J., et al.
RFC 4086: Randomness Requirements for Security, IETF, 2005.
→ Spécifie les exigences de génération de nombres aléatoires pour les systèmes cryptographiques sécurisés.

Normes et documents techniques

- NIST SP 800-90A Rev.1
Recommendation for Random Number Generation Using Deterministic Random Bit Generators
→ Définit les CSPRNG modernes et leurs architectures sécurisées.
- NIST SP 800-38A
Recommendation for Block Cipher Modes of Operation
→ Décrit les modes d'utilisation d'AES (ECB, CBC, CTR, etc.).

Articles et ressources académiques

- Daemen, J., Rijmen, V.
The Design of Rijndael: AES — The Advanced Encryption Standard, Springer, 2002.
→ Explication approfondie de la conception interne d'AES.
- Ferguson, N., Schneier, B., Kohno, T.
Cryptography Engineering, Wiley, 2010.
→ Analyse des bonnes pratiques de conception des systèmes cryptographiques.

Ressources en ligne

- National Institute of Standards and Technology (NIST)
<https://www.nist.gov>
- IETF RFC Repository
<https://www.rfc-editor.org>

- Crypto documentation (PyCryptodome)
<https://pycryptodome.readthedocs.io>
- C 4086: Randomness Requirements for Security